

BootCamp: Entrega Final

Etapa 3 — Trabalho em Equipe, Banco de Dados e Code Review

1. Identificação do Grupo

Nome do Aluno	Papel / Usuário GitHub
Marco Túlio de Sousa Machado	Dono do Repositório central / @marcomachado28
Lucas Cortez Leoi	Colaborador do Repositório / @lucascortezleoi

2. Links de Entrega Obrigatórios

Item	Link / Endereço
Repositório GitHub	https://github.com/marcomachado28/dashboard.climatico-
Front-end Publicado (Vercel)	https://dashboard-climatico-lyart.vercel.app/
Back-end Publicado (Render)	https://dashboard-climatico-b0zc.onrender.com

3. Apresentação do Projeto e Stack Utilizada

O projeto consiste em um **Dashboard Climático & Monitoramento de Emissões**. Ele foi desenvolvido como uma aplicação web full-stack conectada em tempo real com APIs meteorológicas e bancos de dados hospedados na nuvem. A stack tecnológica adotada inclui as seguintes camadas:

- **Back-end:** Construído em Java 21 utilizando o framework **Spring Boot 3.2.4** (com Spring Data JPA para mapeamento objeto-relacional, Hibernate e Spring Web para as APIs REST).
- **Banco de Dados Relacional:** Hospedado na nuvem pelo provedor serverless **Neon.tech (PostgreSQL)**. Conta com tabelas dimensionais estruturadas para geolocalização e sensores, além de uma tabela factual para leituras climatológicas.
- **Front-end:** Desenvolvido com HTML5, CSS3 plano corporativo (Flat Design focado em legibilidade) e JavaScript (ES6) para requisições assíncronas *fetch* e renderização de gráficos em tempo real via **Chart.js**.
- **Integração Externa:** Integração direta via requisição REST com a **OpenWeather API**.

4. Funcionamento e Conectividade em Tempo Real

A grande melhoria adicionada nesta etapa do projeto foi a criação de um serviço agendador automático (**ClimaScheduler**) ativado no bootstrap do Spring Boot via `@EnableScheduling`. Esse serviço consome a API da OpenWeather a cada 10 minutos para uma lista de 9 capitais mundiais importantes (incluindo Brasília, São Paulo, Tóquio, Londres e Nova York) e persiste as medições de Temperatura e Umidade diretamente no banco Neon PostgreSQL na nuvem de forma automática e sem custos de execução.

Com isso, quando o usuário acessa o dashboard hospedado na Vercel, o JavaScript consome as APIs públicas expostas pelo servidor Java hospedado no Render, carregando gráficos históricos instantaneamente atualizados de diversas partes do mundo.

5. Processo de Code Review e Correções Realizadas

O repositório original continha diversas falhas e classes incompletas. Mapeamos as seguintes correções:

1. **Organização em Módulos:** Estruturação da árvore de diretórios Java conforme o padrão Maven (src/main/java/com/clima).
2. **Conserto de Compilação:** Correção de chaves quebradas e declarações de classe apagadas na entidade Clima.java e ajuste na hierarquia do plugin do Spring Boot no pom.xml.
3. **CI com GitHub Actions:** Configuração do workflow maven.yml para compilar e rodar a suíte de testes de integração (MockMvc) usando banco de dados local H2 em memória, garantindo aprovação de Pull Requests de forma isolada.
4. **Ajuste de Imagens Docker:** Correção do Dockerfile para usar imagens oficiais do Eclipse Temurin JDK 21, evitando falhas de build na plataforma de deploy Render.

6. Aprendizados e Conclusões Acadêmicas

O trabalho em equipe via Pull Requests evidenciou a complexidade de manter o código limpo, consistente e livre de conflitos. O aprendizado da migração da memória interna para o banco de dados Neon PostgreSQL nos ensinou a gerenciar dados relacionais complexos em cloud e garantir conexões seguras SSL no Spring Boot. Além disso, a configuração do pipeline de CI/CD e a modelagem do Dockerfile mostraram a relevância de padronizar ambientes de build para evitar erros em produção, encerrando o bootcamp com uma compreensão holística e prática de engenharia de software no mundo real.